

Malware: Self-replicating malware that spreads through networks without user interaction.
Viruses: Malicious code that attaches to legitimate programs and executes when the host runs.
Trojan Horses: Program that looks harmless but secretly performs malicious operations.
Spyware: Malware that secretly collects user activity or personal data without consent.
Ransomware: Malware that locks or encrypts your files and then demands payment (a ransom) to unlock them.
Botnets: Network of infected devices remotely controlled by an attacker.
Phishing: Fraudulent attempt to obtain sensitive information by impersonating trusted entities (often via email or fake websites).
Phishing VS Pharming: Phishing is tricking people directly (social engineering). Example: fake email or website asking you to log in.
Pharming = technical redirection. The attacker changes DNS or host settings so you visit a fake site even if you type the correct URL.

Ten Security Principles

No.	Principle	Summary
1	Least Privilege	Grant minimal access for minimal time.
2	Defense in Depth	Use multiple layers of protection.
3	Secure Failure	Systems should fail safely.
4	Secure the Weakest Link	A system is only as strong as its weakest part.
5	Compartmentalization	Isolate system components to limit damage.
6	Simplicity	Simple systems are easier to secure.
7	Promote Privacy	Protect user data diligently.
8	It's Hard to Hide Secrets	Avoid relying on obscurity.
9	Don't Extend Trust Easily	Be skeptical of all systems, even internal ones.
10	Trust the Community	Rely on well-tested, reviewed public solutions.

Security: Controls who can do what when.
Privacy: Freedom to control access to one's personal information.
Integrity: Consistency and wholeness of data.
Availability: Systems and data must be available to authorized users.

AAA Security Model: Authentication, Authorization, Accounting.
Supporting Concepts: Security Component: Anything that leads to secure.

Password Security: Having all not guarantee security, but secure need an.

Defensive Measures: Multi-factor authentication (SMS, authenticator apps), Strong password policies, Rate-limiting and lockouts, Use password managers.

Information Security Goals: CIA Triad (Confidentiality, Integrity, Availability).

Auditing Concepts: Logging, Record events or stats about system usage & perf., Auditing: Analyze logs to interpret system status clearly.

Auditing Goals: User accountability: Known who my action, Damage assessment: post-mortem evaluation, improve future defenses, Identify security violation causes: find root cause.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

What to audit: Not all data, only policy-relevant info. Log must record: Date/time: timestamp, User: User accountability, Event type: Success/failure, Terminal ID (or auth): identifies where a login or authentication request originated.

Link Layer Security: No confidentiality, integrity, or authentication. Attacks: Sniffing: Monitoring (WireShark), MAC spoofing: attacker manually changes their own MAC address, ARP poisoning: broad term - attacker sends forged ARP replies to corrupt other hosts' ARP tables.

Hardware mechanisms that support Integrity (isolation): Ring levels (0-3) for privilege separation, Segmentation: Memory divided into logical segments (code, data, stack), Paging: Memory divided into fixed-size blocks (pages) mapped by the OS to physical memory.

Encryption Overview: Asymmetric < Symmetric < Hash (fastest). Purpose: Create Integrity & Security when no hardware support.

Classical history: Systeal (Sparta): Transposition cipher - no simple substitution, Caesar cipher (monoalphabetic): Simple substitution, Vigenere cipher: Polyalphabetic cipher.

Stream Cipher: Encrypts bit-by-bit or character-by-character, Depend On Pos: diff position & same char -> may encrypt differently, Block Cipher: Encrypts fixed-size blocks, Construct cipher blocks.

Zero-Knowledge Password Proof (ZKPP): Prove password knowledge without revealing it, Analogy: girl with key through tunnel -> prove she has a key, but never show the key.

Example Scenario: Asymmetric: Public - Private key, Problem: Word Frequency Analysis -> สามารถเดาได้เพราะมีคำที่พบบ่อย.

GRC: The Strategy for Safety, The "GRC" Triad (Strategy): planning and checking the rules, Gap Assessment (The "Checklist")

Vulnerability Assessment (The "Evaluation"): A systematic process to find known weaknesses, Penetration Test (The "Attack"): A manual, authorized simulated attack where a human expert (Ethical Hacker) tries to exploit the weaknesses found in the assessment.

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Pen Test Process: 4 Main Steps, 1. Scope Definition: The "Contract", 2. Information Gathering: The "Stalking", 3. Enumeration: The "Knocking on Doors", 4. Exploitation: The "Break-In".

Network Layer (IP): Best effort: no ordering, retransmission, or security, Attacks: IP spoofing, sniffing, DHCP abuse, Routing (BGP): Routers trust peers - BGP hijacks (e.g. Pakistan-YouTube 2008), Defense: Secure BGP (BGPsec), Concept: IP Cryptography: ใช้ Public Key ในการเข้ารหัส (Chain) เพื่อป้องกันการปลอมแปลง.

Example Scenario: (ส่งข้อมูลจาก AS1 -> AS2 -> AS3), Step 1: AS1 (Origin) เริ่มส่ง, Goal: ประกาศว่า "ฉันเป็นเจ้าของ Prefix นี้ และฉันส่งต่อให้ AS2", Msg1 = "Prefix: 10.0.0.0/24, Target: AS2", Sig1 = PriKey_AS1(hash(Msg1)), Packet Sent to AS2: [Msg1, Sig1, Certificate_AS1]

Step 2: AS2 (Transit) รับมาและส่งต่อจาก AS1, Action 1: ตรวจสอบตัวตน (Identity Check - Chain of Trust), เห็น Certificate_AS1, Check: เห็น PubKey_APNIC (ที่ตัวเองมี) มาตรวจสอบ Certificate_AS1 -> ตัว PubKey_AS1 ออกมาจาก APNIC, ใช้ PubKey_AS1 และ Sig1 ได้ RcvHash, True if RcvHash == hash(Msg), Result: ข้อความถูกต้อง AS1 ส่งและระบุ Target เป็น AS2 จึง

Action 3: ส่งต่อ (Signing), Msg2 = "Target: AS3", Sig2 = PriKey_AS2(hash(concat(Sig1, Msg2))), Packet Sent: [Msg2, Sig2, Certificate_AS1, Certificate_AS2]

Step 3: AS3 (Validator) ปลายทาง, Action: ตรวจสอบตัวตนกลับ, Layer 2 (ที่ AS2): Check Identity: ใช้ PubKey_APNIC เช็ค Certificate_AS2 -> ได้ PubKey_AS2, Check Sig: ใช้ PubKey_AS2 เช็ค Sig2 -> ยืนยันว่า AS2 ส่งให้ AS3

Layer 1 (ที่ AS1): Check Identity: ใช้ PubKey_APNIC เช็ค Certificate_AS1 -> ได้ PubKey_AS1, Check Sig: ใช้ PubKey_AS1 เช็ค Sig1 -> ยืนยันว่า AS1 ส่งให้ AS2

Curiosity Corner: Why Encrypt Hash? Encrypt(hash(Msg)) faster than Encrypt(Msg), Why concat(Sig1, Msg2)? If missing: Hacker can do a Cut-and-Paste Attack (เอา Sig2 ที่ถูกต้องไปแปะกับ Route ขึ้นที่ปลอมขึ้นมา), With Sig: เมื่อมีการ "ยืนยัน" เอกสารดิจิทัล ถ้า Sig1 (ที่แท้จริง) ถูกหักออก หรือเปลี่ยน ค่า Hash จะเปลี่ยน ส่งผลให้ Sig2 (ที่ตรวจสอบ) ผิดเพี้ยน -> ยืนยันว่าเส้นทางต่อเนื่องกันจริง (Unbroken Chain).

Comparison: ทำไม BGP Hijack ถึงทำได้ง่าย?, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Result: Chain จาก AS3 ที่ขึ้นที่ Hacker ไม่ได้รับอนุญาตจาก AS1 -> Drop Packet!, Scenario: Hacker (AS_Evil) ขโมยมาแทรกกลาง, Attack: Hacker รับ [Msg1, Sig1] จาก AS1 มา แล้วแยกเป็นสองส่วน AS3, Problem: Hacker ส่งต่อ Sig_Evil = PriKey_Evil(Sig1 + "Target: AS3")

Address Protection: *Dynamic Solution*

- Canary Words:** มาจาก Canary Bird (นกขมิ้น -> อ่อนแอ ต่างๆ) -> จะถูกแก้ก่อน RET โจนเนอร์
 - ก็ระหว่าง Buffer กับตัวสำคัญ ถ้า overflow Stack ต้องถูกแก้
 - มักใช้กับ "Null Byte" (0x00) อยู่ด้วย เพราะ **strcpy** หยุดทำงานเมื่อเจอ Null Byte
 - "ไม่บั๊ตเช็กลูกขุนเตือน แต่เป็น "กำแพง" ที่ทำให้**strcpy**หยุดที่มันด้วย
 - StackGuard:** Canary word before return address -> Detect Overwrite
 - ปัญหา:** ถ้า hacker รู้ Canary Word ก็แก้ไขแทนให้ถูกก็ได้
- Address Encode:** Encrypt Address ทำให้แฮกเกอร์ไม่ได้ แต่เนื่องจากไม่รู้ว่า Encrypt อย่างไร เมื่อไร Decrypt ก็จะไปหาคำไหนไม่รู้ ซึ่งมีโอกาสสูงจะ Crash -> ทำให้เราไม่ปรับโค้ดที่ hacker อาจกินได้
 - PointerGuard:** Encrypts pointers, Protects function pointers
 - SPEF:** Encrypted instruction/code
 - Instruction Set Randomization (ISR):** เข้ารหัสสุ่มคำสั่งด้วย per-process key (เช่น XOR) ทำให้โค้ดที่รันมีชื่อมาไม่ตรงกับ ISA ที่ถูกถอดรหัสจึง (Columbia / Drexel)
 - ปกติ ADD อาจจะเป็น Opcode 0x01 ในทุกเครื่อง.
 - ISR จะสุ่มให้ Process A ใช้ ADD = 0x55, Process B ใช้ ADD = 0xA6, ซึ่ง hacker เอาไม่ได้อีก
 - ปัญหา:** Performance, Compatibility
- Copy of Address:** เก็บ return address สองที่จริง แล้วจะ ret จะเห็นสองค่าถ้าโดนกรอกก็ถูกเขียนทับ
 - Split Stack:** Separates control(ret)/data(buffer) stack มี ret ที่เดียว แยกไว้เลย
 - ปัญหา:** ทับสองที่ก็โดนออกดี
- Tags:** บิตพิเศษ meta data ที่ติดไปกับแต่ละคำของหน่วยความจำ (memory word) เพื่อบอกว่าให้แสดงว่า "ข้อมูลนี้มาจากที่ไหน" ถ้าจะชี้ตัวก็เป็น Addr แต่ต้นมาจาก input -> ไม่อนุญาต
- Input Protection:** Prevents untrusted data as control data -> Secure bit

Bounds Checking: Validates memory access : ของเขาให้เขียนเท่านั้น — ทางที่ถูกตั้งแต่แพง

- Array Bounds Checking:** Software-level checks
- Segmentation / hardware bounds:** Hardware-level checks
- LibSafe / LibVerify:** ห่อหุ้ม libc ให้ตรวจหา bounds และกรองการเรียก I/O/strcpy ก่อนเขียนจริง (Bell Labs)
- ปัญหา:** ช้าลงได้มาก (ตัวอย่างในโลกความจริงถึง ~30x slowdown ในบางกรณี)

Obfuscation: ASLR (Address Space Layout Randomization) -> เอาไม่ได้อะไร RET อยู่ไหน, Hack ได้แต่ยากขึ้นเฉพาะ การเจอกันแค่ผ่าน Source Code ก็ได้แล้ว **ASLR**

Data Hiding Techniques : Disguises information from normal view

Digital Forensics : make data admissible as evidence in court

- Rename:** Changing name and extension -> make it look harmless/irrelevant
 - Weakness : Forensic tool check file header can compare the extension
- Attributes hiding:** Mark as Hidden in OS -> disappears from normal file explorer view
 - Weakness : Visible if system show hidden files
- Bit-shifting:** Shift binary data to hide content -> file become unreadable (lightweight encryption) -> but easy to crack.
- Encrypt/Password:** Data unreadable without key
- Partition hiding (ไม่ mount):** **diskpart remove letter** -> OS won't detect that partition (OS wont mount), forensic data still exists.
 - Weakness : Visible to forensic tools that scan unallocated space, analyze disk space
- Marking bad clusters (Mount แทนไม่ใช้เพราะถูก mark ว่าเสีย):** FAT file system, attacker can manually mark good clusters as "bad", so the OS skips them, assuming they are damaged.
 - Weakness : Detectable with modern forensic tools but invisible to casual users.

Steganography & Steganalysis

- Steganography:** hiding information inside other files (images, audio).
 - เอาข้อความลับซ่อนไว้ในรูปภาพ โดยเปลี่ยนค่า บิตเล็ก ๆ (LSB – Least Significant Bit) ของพิกเซล
 - ส่งข้อความในไฟล์เสียง/วีดิโอโดยเปลี่ยนค่าที่มนุษย์ฟังไม่ต่าง
 - Keys : Human not noticable to these changes
- Steganalysis:** detecting and analyzing Steganography.
- Digital watermarking:** hides ownership info -> บอกว่าไฟล์นี้ของใคร, พยายามเอาสออด.
Steganography that for license purpose not hiding information เช่น ข้างภาพখনขึ้นชื่อ .jpg
- Authorization:** "What can you do?" -> **Policy will tell you** **AuthZ**
 - Policy (Law) :** Help to decide
 - App-specific decision-making based on policy : Something action is different in different application -> No general AuthZ can provide for every application
 - Type Enforcement (Police) : Enforce to follow policy
- Security Policy:** Clearly define what is secure or insecure, We cannot say something is "safe" unless we define what "safe/unsafe" looks like
- Secure System:** Start Secure + no Insecure Moves = Always Secure.
 - Starts in a secure state
 - Cannot enter an insecure state.

Access Control Models

- Mandatory Access Control (MAC)
 - Admin-defined policies; users untrusted.
 - Multilevel system : Higher Level is king.
- Discretionary Access Control (DAC)
 - Owner-defined policies
 - Common in personal computing/drive.
 - Not combination of MAC & DAC
- Role-Based Access Control (RBAC)
 - Who know about role defined policies
 - Based on least privilege; **access tied to job roles**.
 - No clear definition on how to define a role lead to **Role Explosion**
 - "Nurse" role,
 - "Nurse who works night shift" role
 - "Nurse who works night shift and handles X-rays."
 - Suddenly, you have 1,000 roles, and it's a mess.
- Attribute-Based Access Control (ABAC)
 - Decisions use attributes to map access ability,
 - Attributes : User att, Resource att, Environmental att
- Modern Access Control
 - OAuth 2.0 / OIDC:** Limit-Accessed Pass
 - Zero Trust Architecture:** Never Trust, Always Verify

build around CIA triad

hack C: Monitor, DB I: Modify J: Dosy

(ผู้กำหนดระบบ) กำหนดสิทธิ์ (เช่น เป็นแอดมิน) แอดมินอาจทำให้อุปกรณ์ใดก็ตามสามารถเข้าถึงได้เมื่อ User มี Discretionary Access Control (DAC)

attached to Resource (ทรัพยากร) ที่เข้าถึงได้ทั้งระบบ (tokens/keys) attached to Subject (ผู้ใช้) หรือจะใส่เข้าไปได้

Profile DAC Drive DAC Windows filesystem DAC Registry registration system RBAC

Policy Development Steps

- Analyze security model (Bell-LaPadula, Biba, China Wall).
- Choose access control model (MAC, DAC, RBAC, ABAC).
- Convert policy – Access Control Matrix.
- Implement via Type Enforcement (ACLs, Capabilities).